

ApplyFlow — `architecture.md` (V1)

1. Purpose of This Document

This document defines the **implementation architecture** for ApplyFlow V1.

Its job is to turn `spec.md` into a codebase structure that AI coding agents (Claude, Codex) can implement consistently **without inventing architecture on their own**.

This document optimizes for:

- **clarity over cleverness**
- **clean code over compact code**
- **reviewability over abstraction hype**
- **small-project pragmatism over enterprise ceremony**

The goal is that a reviewer should be able to open the project and quickly answer:

- where each feature lives
 - where business logic lives
 - where MongoDB access happens
 - where attention-flag logic happens
 - what each module is responsible for
 - how to add a new feature without breaking structure
-

2. Architecture Principles

2.1 Primary Principles

ApplyFlow V1 follows these principles:

1. Keep the code easy to read

Prefer:

- explicit names
- small functions
- predictable file locations
- straightforward control flow

Avoid:

- “smart” abstractions that hide behavior
- giant utility files
- controllers doing everything
- deeply nested callbacks or giant service methods

2. Separate HTTP concerns from business logic

Controllers should handle:

- request parsing
- calling service functions
- shaping HTTP response

Controllers should **not** contain:

- complex attention-flag logic
- long MongoDB query construction
- cross-entity business rules
- timeline derivation logic

That belongs in services and repositories.

3. Separate business logic from persistence logic

Business logic answers:

- what should happen
- what rules apply
- what counts as “no response after apply”

Persistence logic answers:

- which collection to query
- how to build MongoDB filters
- how to insert/update documents

These should not be mixed in the same function unless the function is extremely trivial.

4. Prefer explicit modules over generic base classes

For V1, do **not** build:

- abstract repository base classes
- generic CRUD superclasses
- generic “manager” layers
- dynamic magic registries

They add indirection without helping a small project.

Use explicit files like:

- `application.service.js`
- `application.repository.js`

- `event.service.js`
 - `dashboard.service.js`
-

5. Make business rules easy to test

The attention-flag logic is one of the most important parts of the product.

Therefore:

- attention rules must live in a dedicated module
 - rule functions should be as pure as practical
 - date-threshold logic should be centralized
 - services should be testable without spinning up the whole app
-

6. Avoid hidden side effects

A function named `createEvent()` should not silently:

- update unrelated collections
- auto-change status in surprising ways
- create extra reminders unless explicitly defined

V1 should keep side effects narrow and visible.

2.2 Clean Code Rules for AI Contributors

Every AI agent contributing code to this project must follow these rules.

Naming

Use names that describe the business meaning clearly.

Prefer:

- `createApplication`
- `getApplicationByIdForUser`
- `listApplicationEvents`
- `computeAttentionFlags`
- `findUpcomingEventsForUser`

Avoid vague names like:

- `handleData`
- `doThing`
- `processApp`
- `manageEvent`
- `helper`

Function size

A function should usually do **one thing**.

If a function is handling:

- validation normalization
- MongoDB query construction
- business rule evaluation
- response mapping

all at once, it is probably too large.

Controller size

Controllers should stay thin.

If a controller exceeds roughly **30–50 lines of real logic**, it likely needs work moved into services.

File responsibility

Each file should have a clear responsibility.

Good:

- one controller file per domain
- one service file per domain
- one repository file per domain
- one validator file per domain

Bad:

- one giant `utils.js`
 - one `common.service.js` with half the app inside
 - one controller that handles auth, applications, and events
-

Comments

Prefer **clear code first**.

Use comments for:

- non-obvious business rules
- attention-flag edge cases
- reasoning behind a constraint

Do not add comments that merely restate the code.

Bad:

```
```js id="e1" const now = new Date() // get current date
```

Useful:

```
```js id="e2"
// Silence flags should only apply to active applications.
// Rejected / withdrawn cases must not show "no response" reminders.
```

Error handling

Throw domain-relevant errors with meaningful messages.

Do not return vague failures like:

- "Something went wrong"
- "Invalid request" for every case

No “clever” Mongo pipelines unless needed

For V1, prefer understandable query flow over ultra-compressed aggregation wizardry.

If a normal `find()` + in-memory rule evaluation is easier to review and the dataset is small enough, that is acceptable.

3. High-Level System Architecture

ApplyFlow V1 uses a **modular monolith** architecture.

It is one application split into:

- **Frontend:** React + Vite
- **Backend API:** Node + Express
- **Database:** MongoDB

3.1 Logical View

```
```text id="3y6j2t" Frontend (React/Vite) | v HTTP API (Express) | v Route -> Middleware -> Controller ->
Service -> Repository -> MongoDB | +-> Domain logic / attention rules / validators
```

---

### ## 3.2 Why This Architecture

This architecture is chosen because it is:

- small enough for one-person development
- easy for AI coding agents to follow
- easy to review
- flexible enough for future expansion

It avoids:

- premature microservices
- heavy DDD ceremony
- hidden ORM behavior
- tangled controller-driven code

---

## # 4. Technology Decisions

### ## 4.1 Backend

- **Node.js**
- **Express**
- **MongoDB native driver** (``mongodb``)
- JWT-based authentication
- bcrypt for password hashing
- optional validation library such as Zod/Joi, or custom validators if kept consistent

### ## 4.2 Frontend

- **React**
- **Vite**
- React Router
- a lightweight HTTP client (``fetch`` wrapper or axios)
- UI library optional, but architecture should not depend on one

### ## 4.3 Database

- **MongoDB**
- Collections:
  - ``users``
  - ``applications``
  - ``application_events``

---

## # 5. Repository Pattern Choice

### ## 5.1 Decision

ApplyFlow V1 will use a **light repository layer**.

This means:

- repositories contain MongoDB collection access logic
- services contain business rules and orchestration
- controllers stay thin

### ## 5.2 Why not skip repositories entirely?

Skipping repositories would push MongoDB calls into services. That works for very tiny apps, but this project already has:

- auth
- applications
- events
- dashboard summaries
- attention-flag logic

A repository layer gives cleaner separation without too much complexity.

### ## 5.3 Why not a heavy repository abstraction?

Because V1 does not need:

- generic base repositories
- inheritance hierarchies
- abstract factory patterns

Each domain gets an explicit repository file.

---

## # 6. Backend Folder Structure

### ## 6.1 Target Backend Structure

```text id="8u4dpa"

```
backend/  
└─ src/  
    │  
    │├─ app.js  
    │├─ server.js  
    │  
    │├─ config/  
    ││├─ env.js  
    ││├─ mongodb.js  
    ││└─ constants.js  
    │  
    │├─ routes/  
    ││├─ index.js  
    ││├─ auth.route.js  
    ││├─ application.route.js  
    ││├─ event.route.js  
    ││└─ dashboard.route.js  
    │  
    │├─ middlewares/  
    ││├─ auth.middleware.js  
    ││├─ error.middleware.js  
    ││└─ validate.middleware.js
```

```
└─ modules/
  └─ auth/
    │ └─ auth.controller.js
    │ └─ auth.service.js
    │ └─ auth.repository.js
    │ └─ auth.validator.js
    │ └─ auth.mapper.js
    │
    └─ application/
      │ └─ application.controller.js
      │ └─ application.service.js
      │ └─ application.repository.js
      │ └─ application.validator.js
      │ └─ application.mapper.js
      │
      └─ event/
        │ └─ event.controller.js
        │ └─ event.service.js
        │ └─ event.repository.js
        │ └─ event.validator.js
        │ └─ event.mapper.js
        │
        └─ dashboard/
          │ └─ dashboard.controller.js
          │ └─ dashboard.service.js
          │ └─ dashboard.mapper.js
          │
          └─ domain/
            │ └─ attention/
              │ └─ attention.service.js
              │ └─ attention.rules.js
              │ └─ attention.types.js
              │ └─ attention.utils.js
              │
              └─ timeline/
                │ └─ timeline.utils.js
                │
                └─ shared/
                  │ └─ domain-errors.js
                  │
                  └─ db/
                    │ └─ collections.js
                    │ └─ indexes.js
                    │
                    └─ utils/
                      │ └─ async-handler.js
                      │ └─ date.utils.js
                      │ └─ object-id.utils.js
                      │ └─ pagination.utils.js
                      │
                      └─ shared/
```



```
|— api-response.js
|— logger.js
```

7. Backend Layer Responsibilities

7.1 Route Layer

Responsibility: define HTTP endpoints and attach middleware + controllers.

Routes should:

- map URL to controller
- attach auth middleware where needed
- attach validation middleware if used

Routes should **not**:

- contain business logic
- talk to MongoDB
- transform data heavily

Example:

- `application.route.js` defines CRUD routes for applications
 - `event.route.js` defines nested routes under applications for events
-

7.2 Middleware Layer

Responsibility: cross-cutting concerns.

Includes:

- JWT auth verification
- request validation
- global error handling

Middleware should **not**:

- compute attention flags
 - create/update applications
 - contain domain-specific orchestration
-

7.3 Controller Layer

Responsibility: translate HTTP request/response into service calls.

Controllers should:

- read params/query/body/user
- call a service
- send JSON response
- pass errors to next middleware

Controllers should **not**:

- write MongoDB queries
 - compute dashboard summaries
 - evaluate silence flags directly
-

7.4 Service Layer

Responsibility: business logic and orchestration.

Services should:

- enforce domain rules
- call one or more repositories
- coordinate cross-entity workflows
- call attention/timeline domain helpers where needed

Examples:

- `application.service.js`
 - create application
 - update application
 - list applications with filters
 - `event.service.js`
 - ensure application ownership before adding event
 - list application timeline
 - `dashboard.service.js`
 - fetch applications/events
 - compute summary cards
 - compute attention flags
 - compute upcoming events
-

7.5 Repository Layer

Responsibility: all MongoDB read/write access for a domain.

Repositories should:

- get collection references
- build MongoDB queries
- perform inserts/updates/deletes/finds
- return raw or lightly normalized DB data

Repositories should **not**:

- decide if an application deserves `NO_RESPONSE_AFTER_APPLY`
 - compute dashboard sections
 - contain request/response concerns
-

7.6 Domain Layer

Responsibility: shared business logic that is not tied to HTTP or a single repository.

This is where important reusable rules live.

`domain/attention`

Contains all attention-flag logic:

- flag type definitions
- rule evaluation functions
- helper functions for date/event calculations

`domain/timeline`

Contains timeline-specific helpers:

- effective event date calculation
- sorting helpers

`domain/shared`

Contains domain-level error classes if needed.

8. Backend Module-by-Module Design

8.1 Auth Module

Files

- `auth.controller.js`
- `auth.service.js`

- `auth.repository.js`
- `auth.validator.js`
- `auth.mapper.js`

Responsibilities

Controller

Handles:

- `POST /auth/register`
- `POST /auth/login`
- `GET /auth/me`

Service

Handles:

- register user
- validate login credentials
- generate JWT payload/token
- sanitize returned user object

Repository

Handles:

- find user by email
- create user
- find user by id

Validator

Validates:

- register payload
- login payload

Mapper

Transforms user DB document to safe response shape.

8.2 Application Module

Files

- `application.controller.js`
- `application.service.js`
- `application.repository.js`

- `application.validator.js`
- `application.mapper.js`

Responsibilities

Controller

Handles:

- `POST /applications`
- `GET /applications`
- `GET /applications/:applicationId`
- `PATCH /applications/:applicationId`
- `DELETE /applications/:applicationId`

Service

Handles:

- create application for current user
- list user applications with search/filter/sort
- fetch single application ensuring ownership
- update application ensuring ownership
- delete application and cascade delete events

Repository

Handles:

- insert application
- find applications by user with filters
- count by status if needed
- find application by id + user
- update application
- delete application

Validator

Validates:

- create application payload
- update application payload
- query params for list filters if desired

Mapper

Shapes application response objects.

8.3 Event Module

Files

- `event.controller.js`
- `event.service.js`
- `event.repository.js`
- `event.validator.js`
- `event.mapper.js`

Responsibilities

Controller

Handles:

- `POST /applications/:applicationId/events`
- `GET /applications/:applicationId/events`
- `PATCH /applications/:applicationId/events/:eventId`
- `DELETE /applications/:applicationId/events/:eventId`

Service

Handles:

- verify parent application ownership
- create event
- list timeline events for an application
- update event
- delete event

Repository

Handles:

- insert event
- find events by application and user
- find event by id
- update event
- delete event
- delete many by application id (for cascade delete)

Validator

Validates event create/update payloads.

Mapper

Shapes event response objects.

8.4 Dashboard Module

Files

- `dashboard.controller.js`
- `dashboard.service.js`
- `dashboard.mapper.js`

Responsibilities

Controller

Handles:

- `GET /dashboard/summary`

Service

Builds dashboard data:

- status summary counts
- upcoming events
- attention flags

The dashboard module does **not** need its own repository if it can compose:

- application repository
- event repository
- attention domain services

This is intentional.

Dashboard is an orchestration layer, not a primary persistence domain.

9. Domain Layer Design

9.1 Attention Domain

This is one of the most important parts of the codebase.

It should be isolated and readable.

Files

- `attention.service.js`
 - `attention.rules.js`
 - `attention.types.js`
 - `attention.utils.js`
-

9.2 Responsibility Split

`attention.types.js`

Contains constants / enums for attention flag types.

Example:

```
```js id="a1" export const ATTENTION_FLAG_TYPES = { NO_RESPONSE_AFTER_APPLY:  
'NO_RESPONSE_AFTER_APPLY', NO_RESPONSE_AFTER_INTERVIEW: 'NO_RESPONSE_AFTER_INTERVIEW',
FOLLOW_UP_OVERDUE: 'FOLLOW_UP_OVERDUE' }
```

---

### `attention.utils.js`

Contains small helpers such as:

- get effective event date
- check whether date is older than N days
- sort events by effective date
- determine whether an application status is active for silence flags

Example function names:

- `getEventEffectiveDate(event)`
- `isActiveSilenceStatus(status)`
- `daysSince(date, now)`

---

### `attention.rules.js`

Contains **pure-ish rule functions**.

Each rule should be isolated and readable.

Example exported functions:

- `evaluateNoResponseAfterApply(application, events, now)`
- `evaluateNoResponseAfterInterview(application, events, now)`
- `evaluateFollowUpOverdue(application, now)`

Each function should return either:

- `null` if no flag applies
- or a flag object

This is important because it makes unit testing straightforward.

---



```

`attention.service.js`
Orchestrates rule evaluation over many applications.

Example responsibilities:

- group events by application
- run all relevant rule functions
- flatten non-null flags into one array

Possible exported functions:

- `computeAttentionFlags(applications, events, now = new Date())`
- `computeUpcomingEvents(applications, events, now = new Date())`

9.3 Timeline Domain

File
- `timeline.utils.js`

Responsibilities
Shared timeline helpers:

- compute effective event date
- timeline sorting logic
- maybe event-to-display-date formatting helpers if needed on backend

Possible functions:

- `getEventEffectiveDate(event)`
- `compareTimelineEvents(a, b)`

If `getEventEffectiveDate` is used by both timeline and attention logic,
either:

- keep it in `timeline.utils.js` and import from there
- or move it into a shared domain date utility

The key rule is: do not duplicate this logic in multiple places.

10. Database Access Design

10.1 MongoDB Connection

Use a centralized connection setup.

`config/mongodb.js`
Responsibilities:

```

- create MongoClient
- connect once at startup
- export DB instance accessors

Do not open a new connection inside repositories.

---

## # 10.2 Collection Access

## `db/collections.js`

Expose helper functions such as:

- `getUsersCollection()`
- `getApplicationsCollection()`
- `getApplicationEventsCollection()`

This prevents collection-name strings from being scattered across the codebase.

---

## # 10.3 Suggested Indexes

## `users`

- unique index on `email`

## `applications`

Recommended indexes:

- `{ userId: 1, updatedAt: -1 }`
- `{ userId: 1, currentStatus: 1 }`
- text-like search support can be approximated with:
  - `{ userId: 1, company: 1 }`
  - `{ userId: 1, role: 1 }`

If later needed, a more advanced text-search approach can be added.

## `application\_events`

Recommended indexes:

- `{ applicationId: 1, userId: 1 }`
- `{ userId: 1, scheduledAt: 1 }`
- `{ applicationId: 1, type: 1 }`

---

## # 11. API Flow by Feature

### # 11.1 Create Application Flow

```

Route
`POST /applications`

Flow
1. route applies auth middleware
2. route applies request validation
3. controller reads body + authenticated user
4. service validates domain assumptions if needed
5. repository inserts application document
6. service returns created application
7. controller sends response

Notes
The controller should not build the application document directly beyond
passing validated input.

11.2 List Applications Flow

Route
`GET /applications`

Flow
1. auth middleware verifies user
2. controller reads query params:

 - `search`
 - `status`
 - `sortBy`
 - `sortOrder`
3. service normalizes list options
4. repository performs Mongo query
5. service returns list
6. controller sends response

11.3 Create Event Flow

Route
`POST /applications/:applicationId/events`

Flow
1. auth middleware verifies user
2. validation middleware validates event payload
3. controller reads `applicationId`, body, user
4. event service checks parent application exists for this user
5. event repository inserts event
6. service returns created event

```

7. controller sends response

## Important rule

Event creation must not assume the application exists just because an ID was provided.

Ownership check is mandatory.

---

# 11.4 Delete Application Flow

## Route

`DELETE /applications/:applicationId`

## Flow

1. auth middleware verifies user
2. controller passes user + applicationId to service
3. application service ensures application exists and belongs to user
4. application repository deletes application
5. event repository deletes all child events for that application
6. controller returns success response

---

# 11.5 Dashboard Summary Flow

## Route

`GET /dashboard/summary`

## Flow

1. auth middleware verifies user
2. dashboard controller calls dashboard service
3. dashboard service:
  - fetches applications for user
  - fetches relevant events for user
  - computes status summary
  - computes upcoming events
  - computes attention flags
4. dashboard mapper shapes final response
5. controller sends response

---

# 12. Dashboard Computation Strategy

## 12.1 Design Choice

For V1, dashboard summary should be computed in the backend service layer using:

- application repository

- event repository
- attention domain service

## ## 12.2 Why this choice

This keeps the frontend simple and avoids duplicating business rules in the client.

## ## 12.3 Performance tradeoff

For a personal tracker app, user data volume is small enough that a straightforward approach is acceptable:

- fetch user applications
- fetch user events
- compute flags in memory

This is preferable to over-optimizing too early with large aggregation pipelines.

If performance later becomes a problem, the computation strategy can be refined.

---

## # 13. Detailed Responsibility of Key Services

### # 13.1 `application.service.js`

Suggested exported functions:

- `createApplication(userId, payload)`
- `listApplications(userId, queryOptions)`
- `getApplicationById(userId, applicationId)`
- `updateApplication(userId, applicationId, payload)`
- `deleteApplication(userId, applicationId)`

#### ### Responsibilities

- validate ownership assumptions where necessary
- normalize optional fields if needed
- delegate DB operations to repository
- coordinate cascade delete on application deletion

---

### # 13.2 `event.service.js`

Suggested exported functions:

- `createEvent(userId, applicationId, payload)`
- `listApplicationEvents(userId, applicationId)`
- `updateEvent(userId, applicationId, eventId, payload)`
- `deleteEvent(userId, applicationId, eventId)`

```

Responsibilities
- verify parent application ownership
- verify event belongs to application/user
- coordinate event ordering if needed
- delegate DB operations to repository

13.3 `dashboard.service.js`

Suggested exported functions:

- `getDashboardSummary(userId)`

Responsibilities
- fetch user applications
- fetch user events
- compute:
 - `statusCounts`
 - `upcomingEvents`
 - `attentionFlags`

Internal helper functions inside this service may include:
- `buildStatusCounts(applications)`
- `mapApplicationsById(applications)`

Keep these helpers private to the module unless reused elsewhere.

13.4 `attention.service.js`

Suggested exported functions:

- `computeAttentionFlags(applications, events, now = new Date())`
- `computeUpcomingEvents(applications, events, now = new Date())`

Responsibilities
- group events by application
- run rule functions
- return normalized output shape for dashboard use

14. Attention Logic Architecture

14.1 Why isolate it
Attention logic is easy to get wrong because it mixes:

- application status

```

- event history
- date thresholds
- "latest relevant event" reasoning

If it is scattered across controller/service/repository files, review becomes painful.

Therefore all attention rules must live under:

```
- `src/domain/attention/`
```

```

```

## # 14.2 Rule design

Each rule should look conceptually like this:

```
```js id="b3"
function evaluateNoResponseAfterApply(application, events, now) {
  // 1. Check status eligibility
  // 2. Find latest applied event
  // 3. Check for later progress events
  // 4. Check threshold
  // 5. Return flag or null
}
```

Each rule should:

- accept plain JS objects
- avoid DB access
- avoid Express request/response objects
- return predictable output

This makes them easy to test and easy to reason about.

14.3 Attention flag output shape

Recommended internal flag shape:

```
```json id="w4" { "flagType": "NO_RESPONSE_AFTER_APPLY", "applicationId": "app-id", "company":
"OpenAI", "role": "Frontend Intern", "message": "Applied 16 days ago but no response has been
recorded yet.", "referenceDate": "2026-06-17T00:00:00.000Z" }
```

The rule functions may return this final shape directly, or return an intermediate shape that is later mapped.

For V1, returning the final shape directly is acceptable and simpler.

---

## # 15. Validation Strategy

### ## 15.1 Validation Layer

Validation should happen **before service logic** whenever possible.

Use:

- `auth.validator.js`
- `application.validator.js`
- `event.validator.js`

### ## 15.2 What validators should do

Validators should ensure:

- required fields exist
- enums are valid
- dates are parseable
- strings are of expected type/shape

### ## 15.3 What validators should not do

Validators should not:

- query MongoDB for ownership checks
- compute dashboard flags
- enforce cross-collection business logic

That belongs to services.

---

## # 16. Error Handling Strategy

### # 16.1 Error Types

Prefer a small set of explicit error classes, such as:

- `BadRequestError`
- `UnauthorizedError`
- `ForbiddenError`
- `NotFoundError`
- `ConflictError`

These can live in:

- `src/domain/shared/domain-errors.js`

### ## 16.2 Error Flow

- service or middleware throws an error
- `error.middleware.js` converts it to consistent JSON response



### ## 16.3 Important rule

Do not mix:

- returning `{ error: ... }`
- throwing errors
- sending response directly from deep service functions

Use one consistent strategy:

- service throws
- controller/middleware handles response

---

## # 17. Response Mapping Strategy

### ## 17.1 Why use mappers

Mongo documents often contain fields we may not want to expose directly or may need normalization for the client.

Each module can have a mapper file.

Examples:

- user mapper removes password hash
- application mapper ensures stable response shape
- event mapper shapes event payloads

### ## 17.2 Keep mappers simple

Mappers should not contain business rules.

They should mainly shape output.

---

## # 18. Frontend Architecture

### # 18.1 Frontend Goals

The frontend should reflect the backend domain structure so the codebase is easy to navigate.

It should not become a random pile of pages and hooks.

---

### # 18.2 Target Frontend Structure

```
```text id="y9u5ga"
```

```
frontend/
```

```
└─ src/
```

```
    │ main.jsx
```

```
    └─ App.jsx
```

```
|
|
|─ app/
|   │ router.jsx
|   │ providers.jsx
|   └─ query-client.js
|
|─ api/
|   │ http-client.js
|   │ auth.api.js
|   │ application.api.js
|   │ event.api.js
|   └─ dashboard.api.js
|
|─ pages/
|   │ LoginPage/
|   │ │ └─ LoginPage.jsx
|   │ └─ RegisterPage/
|   │ │ └─ RegisterPage.jsx
|   │ └─ DashboardPage/
|   │ │ └─ DashboardPage.jsx
|   │ └─ ApplicationsPage/
|   │ │ └─ ApplicationsPage.jsx
|   │ └─ ApplicationDetailPage/
|   │ │ └─ ApplicationDetailPage.jsx
|
|─ features/
|   │ auth/
|   │ │ └─ components/
|   │ │ └─ hooks/
|   │ │ └─ auth.store.js
|   │ │ └─ auth.utils.js
|   │
|   │ └─ applications/
|   │ │ └─ components/
|   │ │ │ └─ ApplicationList.jsx
|   │ │ │ └─ ApplicationCard.jsx
|   │ │ │ └─ ApplicationForm.jsx
|   │ │ │ └─ ApplicationFilters.jsx
|   │ │ │ └─ StatusBadge.jsx
|   │ │ └─ hooks/
|   │ │ └─ application.utils.js
|   │
|   │ └─ events/
|   │ │ └─ components/
|   │ │ │ └─ EventTimeline.jsx
|   │ │ │ └─ EventItem.jsx
|   │ │ │ └─ EventForm.jsx
|   │ │ └─ hooks/
|   │ │ └─ event.utils.js
|   │
|   └─ dashboard/
```

```
├── components/
│   ├── StatusSummaryCards.jsx
│   ├── UpcomingEventsList.jsx
│   └── AttentionFlagsList.jsx
└── hooks/

├── components/
│   ├── layout/
│   ├── common/
│   └── feedback/
├── hooks/
│   └── useDocumentTitle.js
├── utils/
│   ├── date.utils.js
│   └── storage.utils.js
└── constants/
    └── status.js
```

19. Frontend Design Principles

19.1 Page vs Feature Split

Use:

- `pages/` for route-level screens
- `features/` for reusable domain UI and logic

A page composes feature components.

A feature owns its own components and hooks.

19.2 API Layer

Keep API calls in `src/api/`.

Do not scatter `fetch()` or `axios()` calls inside random components.

Example:

- `application.api.js` contains:
- `getApplications`
- `getApplicationById`
- `createApplication`
- `updateApplication`

- `deleteApplication`
-

19.3 Avoid fat pages

A page should orchestrate layout and data flow, not contain every piece of UI and logic inline.

20. State Management Strategy

20.1 V1 Decision

Use simple state management.

Recommended split:

- **auth state**: a lightweight store/context
- **server data**: fetched via a data-fetching layer or custom hooks
- **form state**: local component state or form library

20.2 Avoid premature global stores

Do not create a huge global store for:

- applications
- events
- dashboard
- forms
- filters
- modal state

Keep state close to where it is used unless there is a strong reason not to.

21. Coding Conventions for MongoDB Native Driver

21.1 Keep ObjectId conversion explicit

When an API param is an ID:

- validate it
- convert it explicitly to `ObjectId`

Do not rely on implicit behavior.

21.2 Keep repository filters readable

Prefer:

```
js id="m1"
const filter = {
  _id: new ObjectId(applicationId),
  userId: new ObjectId(userId)
}
```

over compressed one-liners that are harder to review.

21.3 Normalize date handling

Whenever a field is treated as a date:

- validate it on input
- store it consistently as a `Date`
- avoid mixed string/date behavior inside services

22. Suggested Implementation Order

Phase 1 — Project Foundation

Implement:

- backend app bootstrap
- MongoDB connection
- error middleware
- auth middleware
- basic response helpers
- frontend app shell and router

Phase 2 — Auth

Implement:

- register
- login
- me
- auth store/context
- protected routes

Phase 3 — Applications

Implement:

- application repository/service/controller
- create/list/detail/update/delete
- applications page + form

Phase 4 — Events

Implement:

- event repository/service/controller
- timeline UI
- event create/edit/delete flow

Phase 5 — Dashboard

Implement:

- dashboard service
- attention domain
- upcoming events
- status counts
- dashboard page

Phase 6 — Cleanup

Implement:

- validation hardening
- edge-case handling
- consistent empty states
- README
- optional seed data

23. AI Collaboration Rules for This Project

This section is specifically for AI-assisted development.

23.1 Role split

If one AI writes code, the other should review/test, not rewrite architecture on its own.

Example:

- Claude writes backend application module

- Codex reviews:
- folder placement
- naming consistency
- edge cases
- validation coverage
- status/flag logic

or vice versa.

23.2 AI contributors must not change architecture silently

If an AI wants to introduce:

- a new top-level folder
- a new architectural layer
- a new dependency that changes structure
- auto-sync status logic
- background jobs / schedulers
- ORM / Mongoose
- state-management overhaul

it must be treated as an explicit design change, not a casual implementation detail.

23.3 Review checklist for AI-generated code

When one AI reviews another AI's code, it should check:

Structure

- Is the file placed in the correct module?
- Is controller logic too fat?
- Is MongoDB logic leaking into controller?

Naming

- Are functions and variables explicit?
- Are vague names avoided?

Responsibility

- Does each file have a clear job?
- Is business logic sitting in the correct layer?

Validation

- Are enums/dates/required fields validated?

Ownership & security

- Are user-scoped queries enforced?

- Are application/event ownership checks present?

Attention logic

- Are silence flags only applied to eligible statuses?
- Are thresholds and effective dates handled correctly?

Deletion behavior

- Does deleting an application also delete child events?

Output shape

- Are responses consistent and safe?
-

24. Non-Goals of the Architecture

This architecture intentionally does **not** optimize for:

- multi-tenant enterprise scale
- plugin systems
- CQRS/event sourcing
- offline-first sync
- background job orchestration
- full calendar integration
- complex caching layers

Those would distort V1.

25. Final Architectural Summary

ApplyFlow V1 should be implemented as a **modular monolith** with a **layered backend** and **feature-structured frontend**.

Backend stack

- Node + Express
- MongoDB native driver
- modules for auth / application / event / dashboard
- domain layer for attention rules and timeline helpers

Backend rule of thumb

- **Route:** endpoint wiring
- **Middleware:** auth / validation / errors
- **Controller:** request in, response out
- **Service:** business logic

- **Repository:** MongoDB access
- **Domain:** reusable rules like attention logic

Frontend rule of thumb

- **pages** for route screens
- **features** for domain UI + hooks
- **api** for HTTP calls
- keep state simple and local where possible

Most important clean-code constraints

- do not hide business logic in controllers
- do not hide MongoDB logic in random utilities
- do not scatter attention rules across files
- do not add abstractions that make a small app harder to read
- optimize for a reviewer being able to understand the code quickly